CAMPUS NETWORK OPERATIONS

# CampusShield Command Handbook

A practical copy-paste guide for experiments, validation, dashboard operation, evidence analysis, and recovery in Ubuntu/WSL.

**Project path**

```
/mnt/d/CNDC Project/CampusShield
```

> **Most important rule**
>
> Run only one Mininet experiment at a time. Wait for the cleanup messages before starting another experiment.

**Prepared for:** live project demonstrations and repeatable evidence collection
**Environment:** Ubuntu/WSL2 with the CampusShield virtual environment
**Dashboard:** http://localhost:8501

# 1. Start here

Open Ubuntu/WSL. Activate the project environment before commands that use Streamlit, pandas, matplotlib, or the experiment runner.

```
source ~/.venvs/campusshield/bin/activate
cd "/mnt/d/CNDC Project/CampusShield"
```

If the prompt already starts with `(campusshield)`, the environment is active.

## Quick experiment selector

| Goal | Copy-paste command | Produces |
|---|---|---|
| Experiment 1<br>Healthy baseline | `sudo "$(which python3)" experiments/experiment_runner.py baseline --duration 30` | SQLite run,<br>CSV, PNG |
| Experiment 2<br>ANRE OFF vs ON | `sudo "$(which python3)" experiments/experiment_runner.py congestion --duration 20 --repetitions 3` | SQLite runs,<br>comparison CSV/PNG |
| Experiment 3<br>Failure recovery | `sudo "$(which python3)" experiments/experiment_runner.py failure --repetitions 3 --timeout 20` | SQLite runs,<br>recovery CSV/PNG |
| Analyze<br>Experiments 2 and 3 | `python3 experiments/analyze_results.py` | Summary CSV,<br>Markdown, PNG |

**Why sudo uses which python3**

Mininet needs root access, while the active virtual environment supplies matplotlib, pandas, and other project packages. This form passes the environment's exact Python interpreter to sudo.

## Start the dashboard

```
source ~/.venvs/campusshield/bin/activate
cd "/mnt/d/CNDC Project/CampusShield"
streamlit run dashboard/app.py
```

Open **http://localhost:8501** and keep that terminal running. Stop Streamlit with `Ctrl+C`. After an experiment completes, open the sidebar, click **Reload evidence**, and select the newest run.

**Run numbering**

SQLite assigns the run ID. The newest run may not have the number you predicted if validation or another experiment has already inserted a record.

# 2. Core experiments

## Experiment 1 - Healthy baseline

```
cd "/mnt/d/CNDC Project/CampusShield"
sudo "$(which python3)" experiments/experiment_runner.py baseline --duration 30
```

**What it does:** Builds a fresh Mininet/FRR topology, waits for OSPF, generates a controlled 10 Mbps emergency flow, and measures RTT, packet loss, jitter, throughput, utilization, and Network Health Score.

> **Expected final message**
>
> `BASELINE EXPERIMENT PASSED`

**Generated evidence:** `results/baseline_run_<id>.csv` and `results/baseline_run_<id>.png`, plus the full SQLite evidence.

**Parameter guidance:** Use 30 seconds for reportable evidence. The runner accepts a minimum of 3 seconds. Repeating the command creates a new run; it does not overwrite the previous one.

## Experiment 2 - Congestion: ANRE OFF versus ANRE ON

```
source ~/.venvs/campusshield/bin/activate
cd "/mnt/d/CNDC Project/CampusShield"
sudo "$(which python3)" experiments/experiment_runner.py congestion --duration 20 --repetitions 3
```

**What it does:** Runs matched congestion scenarios with ANRE disabled and enabled. It measures real controller state changes, QoS behavior, flow performance, telemetry, and response time without forcing an improvement.

> **Expected final message**
>
> `CONGESTION COMPARISON EXPERIMENT PASSED`

**Generated evidence:** `results/congestion_comparison_<group-id>.csv` and `.png`, plus paired SQLite runs.

**Parameter guidance:** Use three paired repetitions for reportable evidence. Duration is per OFF or ON scenario and must be at least 8 seconds.

# 2. Core experiments - continued

## Experiment 3 - Physical link failure and OSPF recovery

```
source ~/.venvs/campusshield/bin/activate
cd "/mnt/d/CNDC Project/CampusShield"
sudo "$(which python3)" experiments/experiment_runner.py failure --repetitions 3 --timeout 20
```

**What it does:** Verifies the direct R1-R2 route, disables the physical link, measures convergence to R1-R3-R2, restores the link, and measures direct-route restoration, lost probes, and the longest interruption gap.

> **Expected final message**
>
> `LINK FAILURE EXPERIMENT PASSED`

**Generated evidence:** `results/failure_recovery_<group-id>.csv` and `.png`, plus per-repetition SQLite evidence.

**Parameter guidance:** Use three repetitions for reportable evidence. Timeout is the maximum wait for each route change, not the total experiment duration.

## Analyze Experiments 2 and 3

Run this only after collecting at least three complete congestion pairs and three failure repetitions.

```
source ~/.venvs/campusshield/bin/activate
cd "/mnt/d/CNDC Project/CampusShield"
python3 experiments/analyze_results.py
```

Calculates means and sample standard deviations, evaluates each hypothesis from the measured direction of change, and creates the final comparison artifacts.

> **Expected final message**
>
> `CORE EXPERIMENT ANALYSIS PASSED`

- `results/core_experiment_summary.csv`
- `results/core_experiment_analysis.md`
- `results/core_experiment_summary.png`

> **Interpret PASS correctly**
>
> A PASS message confirms that data collection or analysis completed. The measured values - not the PASS label - determine whether a research hypothesis is supported.

# 3. Validation and subsystem commands

| Purpose | Command | What it validates |
|---|---|---|
| Foundation and failover | `sudo python3 topology/campus_topology.py --test` | OSPF, addressing, routes, connectivity, backup path |
| Traffic generation | `sudo python3 topology/campus_topology.py --traffic-test --duration 10` | Emergency, academic, and bulk flows together |
| Static QoS | `sudo python3 topology/campus_topology.py --qos-test --duration 8` | 20/50/30 Mbps classification and contention |
| Telemetry | `sudo python3 topology/campus_topology.py --telemetry-test` | Rates, utilization, RTT, loss, jitter, route |
| Baseline and NHS | `sudo python3 topology/campus_topology.py --baseline-test` | Healthy calibration, thresholds, normalized NHS |
| ANRE controller | `sudo python3 topology/campus_topology.py --anre-test` | Hysteresis and live dynamic QoS actions |
| Adaptive routing | `sudo python3 topology/campus_topology.py --routing-test` | Link health, OSPF cost, hold-down logic |
| Security isolation | `sudo python3 topology/campus_topology.py --security-test` | Namespaced firewall policy with real packets |
| SQLite persistence | `python3 database/validate_database.py` | Schema, WAL, writes, reads, transactions |
| Dashboard check | `python3 dashboard/validate_dashboard.py` | Non-browser dashboard validation |

**Validation data warning**

The SQLite validation command writes a small synthetic run with is_experimental = 0. It tests persistence and must not be presented as experimental evidence.

# 4. Interactive inspection

## Open the Mininet command line

```
cd "/mnt/d/CNDC Project/CampusShield"
sudo python3 topology/campus_topology.py
```

At the `mininet>` prompt, copy one command at a time:

```
nodes
net
pingall
r1 ip route
r1 tc qdisc show dev r1-eth0
r1 tc class show dev r1-eth0
r1 tc filter show dev r1-eth0 parent 10:
r1 tc -s class show dev r1-eth0
r1 vtysh --vty_socket /tmp/campusshield-frr/r1/vty -c "show ip ospf neighbor"
```

Use `exit` to stop the topology.

## Open Mininet with static QoS applied

```
sudo python3 topology/campus_topology.py --enable-qos
```

## Inspect stored database runs

```
cd "/mnt/d/CNDC Project/CampusShield"
sqlite3 database/campusshield.db
```

Then enter:

```
.tables
SELECT id, name, run_type, is_experimental, status FROM experiment_runs;
.quit
```

# 5. Recommended professor demonstration

This sequence produces fresh evidence in front of the examiner while keeping the presentation short and understandable.

| | |
|---|---|
| 01 | **Start the dashboard**<br>Use Terminal 1, open http://localhost:8501, and keep Streamlit running. |
| 02 | **Explain stored evidence**<br>Show one completed run and state that the dashboard reads SQLite and generated result files; it does not fabricate live values. |
| 03 | **Generate a fresh run**<br>Use Terminal 2 and start Experiment 1 with a 30-second duration. |
| 04 | **Narrate live measurements**<br>Point out the RTT, loss, jitter, utilization, throughput, and Network Health Score samples printed in the terminal. |
| 05 | **Wait for completion**<br>Do not switch evidence until BASELINE EXPERIMENT PASSED and the Mininet cleanup messages appear. |
| 06 | **Load the new evidence**<br>In the sidebar, click Reload evidence and select the newest BASELINE run. |
| 07 | **Connect features to evidence**<br>Open Telemetry, Routing, QoS, ANRE, Experiments, Events, and System to show how the measurements, decisions, and stored results relate. |

**Best live choice**

Experiment 1 is the best classroom demonstration because it completes in about 30 seconds and immediately produces a complete new run, CSV, graph, and dashboard evidence.

# 6. Troubleshooting and safe operation

## ModuleNotFoundError under sudo

Use the active virtual environment's exact Python interpreter:

```
sudo "$(which python3)" experiments/experiment_runner.py baseline --duration 30
```

## Stale Mininet state after an interruption

```
sudo mn -c
```

Run this after an interrupted or failed topology. Do not use it while a valid experiment is active.

## Open vSwitch service is stopped

```
sudo systemctl enable --now openvswitch-switch
```

## Dashboard still shows the older run

Wait for the experiment to finish, open the sidebar, click **Reload evidence**, and select the newest run. The dashboard intentionally does not jump to a new run while evidence is still being written.

## Operating rules

- Run project commands in Ubuntu/WSL, not Windows PowerShell.
- Keep only one experiment runner active at a time.
- Wait for Mininet cleanup messages before closing the experiment terminal.
- Use `Ctrl+C` to stop Streamlit.
- Do not start the Streamlit dashboard with sudo.
- Use 30 seconds and three repetitions for reportable evidence, unless the project methodology is intentionally changed.

> **Ready-to-demo checklist**
>
> Open vSwitch running - virtual environment active - dashboard open - no other Mininet job active - professor-facing browser visible - command handbook ready in a second window.